TRABALHO DE BANCO DE DADOS · 4º BIMESTRE

Biblioteca Digital

Stored Procedures, Cursores e Triggers no MySQL

Integrantes	Gustavo Vianna, Gabriel Cardoso, Rafael de Almeida
SGBD	MySQL 8.0 · MySQL Workbench
Scripts	`scripts/4bim/`

1. O projeto

Biblioteca Digital controla o acervo e os empréstimos de uma biblioteca. Neste bimestre as regras de negócio passaram a ser garantidas pelo próprio banco.

Tabela	Papel
`usuarios`	Leitores (localizados pelo e-mail).
`livros`	Acervo, com `qtd_exemplares` e `qtd_disponivel` (estoque mantido pelas triggers).
`emprestimos`	Empréstimos: datas e status (pendente / atrasado / devolvido).
`log_auditoria` *(nova)*	Histórico gravado pelas triggers de auditoria.

Principais regras

- Usuário com atraso ou com 3 empréstimos ativos não pega outro livro; prazo de 1 a 30 dias.
- Só empresta se houver exemplar; o estoque baixa e volta sozinho.
- Multa de R$ 1,50 por dia de atraso (máximo R$ 45,00).
- Devolução não pode ser desfeita; empréstimo aberto não pode ser excluído.
- Toda alteração em empréstimos e preços fica no log.

Integrante	Responsável por
Gustavo Vianna	Empréstimo, resumo do usuário, triggers de INSERT
Gabriel Cardoso	Devolução, renovação, triggers de UPDATE
Rafael de Almeida	Relatório, cursor, triggers de DELETE e de livros, tabela de log

2. Stored procedures

Procedure	O que faz	Parâmetros
`sp_registrar_emprestimo`	Cadastra empréstimo com validação	IN e-mail, título, dias · OUT id
`sp_registrar_devolucao`	Dá baixa e calcula a multa	IN id · OUT multa
`sp_relatorio_emprestimos_usuario`	Relatório por período	IN e-mail, data início, data fim
`sp_resumo_usuario`	Diz se o leitor pode pegar livro	IN e-mail · 3× OUT
`sp_renovar_emprestimo`	Prorroga o prazo (máx. 14 dias)	IN id · INOUT dias

2.1 sp_registrar_emprestimo

Busca usuário e livro, testa as regras com `IF / ELSEIF` e, se algo falhar, para com `SIGNAL`. Se passar, faz o `INSERT` e devolve o id pelo `OUT`.

```
CREATE PROCEDURE sp_registrar_emprestimo(
    IN  p_email         VARCHAR(100),
    IN  p_titulo        VARCHAR(255),
    IN  p_dias_prazo    INT,
    OUT p_id_emprestimo CHAR(36)
)
BEGIN
    DECLARE v_id_usuario CHAR(36);
    DECLARE v_id_livro   CHAR(36);
    DECLARE v_ativos     INT DEFAULT 0;
    DECLARE v_atrasados  INT DEFAULT 0;
    DECLARE v_id_novo    CHAR(36);
    DECLARE v_msg        VARCHAR(255);
    DECLARE c_limite     INT DEFAULT 3;

    SELECT id_usuario INTO v_id_usuario
    FROM usuarios
    WHERE email = p_email;

    SELECT id_livro INTO v_id_livro
    FROM livros
    WHERE titulo = p_titulo;

    -- Atrasado = status 'atrasado' OU pendente com prazo ja vencido
    SELECT COUNT(*),
           COALESCE(SUM(status = 'atrasado'
                        OR data_devolucao_prevista < CURDATE()), 0)
      INTO v_ativos, v_atrasados
    FROM emprestimos
    WHERE id_usuario = v_id_usuario
      AND status <> 'devolvido';

    IF v_id_usuario IS NULL THEN
        SET v_msg = CONCAT('Usuario nao encontrado: ', IFNULL(p_email, 'NULL'));
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_msg;
    ELSEIF v_id_livro IS NULL THEN
        SET v_msg = CONCAT('Livro nao encontrado: ', IFNULL(p_titulo, 'NULL'));
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_msg;
    ELSEIF p_dias_prazo IS NULL OR p_dias_prazo NOT BETWEEN 1 AND 30 THEN
```

```
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Prazo invalido: informe de 1 a 30 dias';
    ELSEIF v_atrasados > 0 THEN
        SET v_msg = CONCAT('Usuario bloqueado: possui ', v_atrasados,
                           ' emprestimo(s) em atraso');
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_msg;
    ELSEIF v_ativos ⩾ c_limite THEN
        SET v_msg = CONCAT('Limite de ', c_limite, ' emprestimos ativos atingido');
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_msg;
    END IF;

    SET v_id_novo = UUID();

    INSERT INTO emprestimos (id_emprestimo, id_livro, id_usuario, data_saida,
                             data_devolucao_prevista, status)
    VALUES (v_id_novo, v_id_livro, v_id_usuario, CURDATE(),
            DATE_ADD(CURDATE(), INTERVAL p_dias_prazo DAY), 'pendente');

    SET p_id_emprestimo = v_id_novo;
END$$
```

P1-valido **Valido: Sofia pega "Dom Casmurro" por 14 dias**

```
CALL sp_registrar_emprestimo('sofia.cardoso@email.com', 'Dom Casmurro', 14, @id_emp);
SELECT @id_emp AS id_emprestimo_gerado;
```

✓ CALL: executado com sucesso

id_emprestimo_gerado
03f773ca-cda7-11f1-8a73-26a4eaeb17d5

1 linha(s)

P1-depois **Depois: emprestimo criado e estoque baixado pela trigger**

```
SELECT l.titulo, e.data_saida, e.data_devolucao_prevista, e.status
FROM emprestimos e
JOIN livros l ON l.id_livro = e.id_livro
WHERE e.id_emprestimo = @id_emp;

SELECT titulo, qtd_exemplares, qtd_disponivel
FROM livros WHERE titulo = 'Dom Casmurro';
```

titulo	data_saida	data_devolucao_prevista	status
Dom Casmurro	2026-10-21	2026-11-04	pendente

1 linha(s)

titulo	qtd_exemplares	qtd_disponivel
Dom Casmurro	3	0

1 linha(s)

O estoque caiu 1 sem nenhum UPDATE na procedure: quem baixou foi a trigger.

P1-inv-bloqueado **Invalido: Ana tem emprestimos em atraso**

```
CALL sp_registrar_emprestimo('ana.silva@email.com', 'Quincas Borba', 14, @id_emp2);
```

× Error Code: 1644. Usuario bloqueado: possui 2 emprestimo(s) em atraso

2.2 sp_registrar_devolucao (OUT)

Calcula os dias de atraso e usa `CASE` para a multa, que volta pelo parâmetro `OUT`.

```
CREATE PROCEDURE sp_registrar_devolucao(
    IN  p_id_emprestimo CHAR(36),
    OUT p_multa         DECIMAL(8, 2)
)
BEGIN
    DECLARE v_status      VARCHAR(10);
    DECLARE v_prevista    DATE;
    DECLARE v_dias_atraso INT DEFAULT 0;
    DECLARE c_valor_dia   DECIMAL(8, 2) DEFAULT 1.50;

    SELECT status, data_devolucao_prevista
      INTO v_status, v_prevista
    FROM emprestimos
    WHERE id_emprestimo = p_id_emprestimo;

    IF v_status IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Emprestimo nao encontrado';
    ELSEIF v_status = 'devolvido' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Este emprestimo ja foi devolvido';
    END IF;

    SET v_dias_atraso = GREATEST(DATEDIFF(CURDATE(), v_prevista), 0);

    SET p_multa = CASE
                      WHEN v_dias_atraso = 0  THEN 0
                      WHEN v_dias_atraso ⩽ 30 THEN v_dias_atraso * c_valor_dia
                      ELSE 30 * c_valor_dia
                  END;

    -- data_devolucao_real e preenchida pela trigger BEFORE UPDATE
    UPDATE emprestimos
    SET status = 'devolvido'
    WHERE id_emprestimo = p_id_emprestimo;
END$$
```

P2-valido Valido: registrar a devolucao e ler a multa pelo OUT

```
CALL sp_registrar_devolucao(@id_bruno, @multa);
SELECT @multa AS multa_a_pagar;
```

✓ CALL: executado com sucesso

multa_a_pagar
36.00

1 linha(s)

P2-inv-repetido Invalido: devolver o mesmo emprestimo de novo

```
CALL sp_registrar_devolucao(@id_bruno, @multa);
```

× Error Code: 1644. Este emprestimo ja foi devolvido

2.3 sp_relatorio_emprestimos_usuario

Consulta com parâmetros: devolve os empréstimos do leitor no período, com situação e multa.

```
CREATE PROCEDURE sp_relatorio_emprestimos_usuario(
    IN p_email       VARCHAR(100),
    IN p_data_inicio DATE,
    IN p_data_fim    DATE
)
BEGIN
    DECLARE v_id_usuario CHAR(36);

    SELECT id_usuario INTO v_id_usuario
    FROM usuarios
    WHERE email = p_email;

    IF v_id_usuario IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Usuario nao encontrado';
    ELSEIF p_data_inicio IS NULL OR p_data_fim IS NULL OR p_data_inicio > p_data_fim THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Periodo invalido: data inicial deve ser menor ou igual a final';
    END IF;

    SELECT l.titulo,
           e.data_saida,
           e.data_devolucao_prevista AS prevista,
           e.data_devolucao_real     AS devolvido_em,
           CASE
               WHEN e.status = 'devolvido' AND e.data_devolucao_real > e.data_devolucao_prevista
                   THEN 'Devolvido com atraso'
               WHEN e.status = 'devolvido'                      THEN 'Devolvido no prazo'
               WHEN e.data_devolucao_prevista < CURDATE()       THEN 'Em atraso'
               ELSE 'No prazo'
           END AS situacao,
           LEAST(GREATEST(DATEDIFF(IFNULL(e.data_devolucao_real, CURDATE()),
                                   e.data_devolucao_prevista), 0), 30) * 1.50 AS multa
    FROM emprestimos e
    JOIN livros l ON l.id_livro = e.id_livro
    WHERE e.id_usuario = v_id_usuario
      AND e.data_saida BETWEEN p_data_inicio AND p_data_fim
    ORDER BY e.data_saida;
END$$
```

P3-valido Valido: historico da Ana nos ultimos 120 dias

```
CALL sp_relatorio_emprestimos_usuario('ana.silva@email.com',
                                      DATE_SUB(CURDATE(), INTERVAL 120 DAY),
                                      CURDATE());
```

titulo	data_saida	prevista	devolvido_em	situacao	multa
Vidas Secas	2026-07-13	2026-07-27	2026-07-26	Devolvido no prazo	0.00
Dom Casmurro	2026-08-22	2026-09-06	2026-09-03	Devolvido no prazo	0.00
Lavoura Arcaica	2026-09-11	2026-09-25	*NULL*	Em atraso	39.00
Vidas Secas	2026-09-26	2026-10-10	*NULL*	Em atraso	16.50
Dom Casmurro	2026-10-01	2026-10-15	2026-10-13	Devolvido no prazo	0.00
O Tempo e o Vento	2026-10-20	2026-11-10	*NULL*	No prazo	0.00

6 linha(s)

P3-inv-periodo Invalido: data inicial maior que a final

```
CALL sp_relatorio_emprestimos_usuario('ana.silva@email.com', CURDATE(),
                                      DATE_SUB(CURDATE(), INTERVAL 30 DAY));
```

✗ Error Code: 1644. Periodo invalido: data inicial deve ser menor ou igual a final

2.4 sp_resumo_usuario (vários OUT)

Não devolve tabela: devolve três valores em variáveis (ativos, atrasados e situação).

```
CREATE PROCEDURE sp_resumo_usuario(
    IN  p_email         VARCHAR(100),
    OUT p_qtd_ativos    INT,
    OUT p_qtd_atrasados INT,
    OUT p_situacao      VARCHAR(20)
)
BEGIN
    DECLARE v_id_usuario CHAR(36);

    SELECT id_usuario INTO v_id_usuario
    FROM usuarios
    WHERE email = p_email;

    IF v_id_usuario IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Usuario nao encontrado';
    END IF;

    SELECT COUNT(*),
           COALESCE(SUM(status = 'atrasado'
                        OR data_devolucao_prevista < CURDATE()), 0)
      INTO p_qtd_ativos, p_qtd_atrasados
    FROM emprestimos
    WHERE id_usuario = v_id_usuario
      AND status <> 'devolvido';

    SET p_situacao = CASE
                         WHEN p_qtd_atrasados > 0 THEN 'BLOQUEADO'
                         WHEN p_qtd_ativos >= 3   THEN 'LIMITE ATINGIDO'
                         ELSE 'LIBERADO'
                     END;
END$$
```

P4-valido Valido: situacao da Sofia e da Ana

```
CALL sp_resumo_usuario('sofia.cardoso@email.com', @ativos, @atrasados, @situacao);
SELECT 'Sofia' AS usuario, @ativos AS ativos, @atrasados AS atrasados, @situacao AS situacao;

CALL sp_resumo_usuario('ana.silva@email.com', @ativos, @atrasados, @situacao);
SELECT 'Ana' AS usuario, @ativos AS ativos, @atrasados AS atrasados, @situacao AS situacao;
```

✓ CALL: executado com sucesso

usuario	ativos	atrasados	situacao
Sofia	1	0	LIBERADO

1 linha(s)

✓ CALL: executado com sucesso

usuario	ativos	atrasados	situacao
Ana	3	2	BLOQUEADO

1 linha(s)

2.5 sp_renovar_emprestimo (INOUT)

O mesmo parâmetro entra com os dias pedidos e sai com os dias concedidos.

```
CREATE PROCEDURE sp_renovar_emprestimo(
    IN    p_id_emprestimo CHAR(36),
    INOUT p_dias          INT
)
BEGIN
    DECLARE v_status   VARCHAR(10);
    DECLARE v_prevista DATE;
    DECLARE c_max_dias INT DEFAULT 14;

    SELECT status, data_devolucao_prevista
      INTO v_status, v_prevista
    FROM emprestimos
    WHERE id_emprestimo = p_id_emprestimo;

    IF v_status IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Emprestimo nao encontrado';
    ELSEIF v_status <> 'pendente' OR v_prevista < CURDATE() THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Renovacao negada: emprestimo devolvido ou em atraso';
    ELSEIF p_dias IS NULL OR p_dias ⩽ 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Informe uma quantidade de dias positiva';
    ELSEIF p_dias > c_max_dias THEN
        SET p_dias = c_max_dias;
    END IF;

    UPDATE emprestimos
    SET data_devolucao_prevista = DATE_ADD(v_prevista, INTERVAL p_dias DAY)
    WHERE id_emprestimo = p_id_emprestimo;
END$$
```

P5-valido **Valido: pede 20 dias, o limite e 14 (INOUT devolve 14)**

```
SET @dias = 20;
CALL sp_renovar_emprestimo(@id_emp, @dias);
SELECT @dias AS dias_concedidos;
```

✓ CALL: executado com sucesso

dias_concedidos
14

1 linha(s)

Pediu 20 dias, recebeu 14 (o limite).

3. Cursor – sp_atualizar_atrasos

Rotina diária: percorre, um a um, os empréstimos *pendentes* já vencidos, marca cada um como *atrasado* e calcula a multa.

DECLARE CURSOR → HANDLER NOT FOUND → OPEN → FETCH em LOOP → LEAVE → CLOSE

```
CREATE PROCEDURE sp_atualizar_atrasos(
    IN  p_data_referencia DATE,
    OUT p_qtd_atualizados INT,
    OUT p_multa_total     DECIMAL(10, 2)
)
BEGIN
    DECLARE v_fim         BOOLEAN DEFAULT FALSE;
    DECLARE v_id          CHAR(36);
    DECLARE v_prevista    DATE;
    DECLARE v_titulo      VARCHAR(255);
    DECLARE v_usuario     VARCHAR(100);
    DECLARE v_dias        INT;
    DECLARE v_multa       DECIMAL(8, 2);
    DECLARE v_data_ref    DATE;

    -- 1) Declaracao do cursor: o conjunto de linhas a percorrer
    DECLARE cur_vencidos CURSOR FOR
        SELECT e.id_emprestimo, e.data_devolucao_prevista, l.titulo, u.nome
        FROM emprestimos e
        JOIN livros   l ON l.id_livro   = e.id_livro
        JOIN usuarios u ON u.id_usuario = e.id_usuario
        WHERE e.status = 'pendente'
          AND e.data_devolucao_prevista < v_data_ref
        ORDER BY e.data_devolucao_prevista;

    -- 2) Handler: quando o FETCH nao encontrar mais linhas, liga v_fim
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fim = TRUE;

    SET v_data_ref = IFNULL(p_data_referencia, CURDATE());

    IF v_data_ref > CURDATE() THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Data de referencia nao pode estar no futuro';
    END IF;

    SET p_qtd_atualizados = 0;
    SET p_multa_total     = 0;

    DROP TEMPORARY TABLE IF EXISTS tmp_atrasos_processados;
    CREATE TEMPORARY TABLE tmp_atrasos_processados (
        usuario      VARCHAR(100),
        livro        VARCHAR(255),
        prevista     DATE,
        dias_atraso  INT,
        multa        DECIMAL(8, 2)
    );

    -- 3) Abertura: executa o SELECT do cursor
    OPEN cur_vencidos;

    leitura: LOOP
        -- 4) Leitura de uma linha por vez
        FETCH cur_vencidos INTO v_id, v_prevista, v_titulo, v_usuario;
```

```
        -- 5) Saida do laco quando acabaram as linhas
        IF v_fim THEN
            LEAVE leitura;
        END IF;

        SET v_dias  = DATEDIFF(v_data_ref, v_prevista);
        SET v_multa = LEAST(v_dias, 30) * 1.50;

        UPDATE emprestimos
        SET status = 'atrasado'
        WHERE id_emprestimo = v_id;

        INSERT INTO tmp_atrasos_processados
        VALUES (v_usuario, v_titulo, v_prevista, v_dias, v_multa);

        SET p_qtd_atualizados = p_qtd_atualizados + 1;
        SET p_multa_total     = p_multa_total + v_multa;
    END LOOP leitura;

    -- 6) Fechamento: libera o cursor
    CLOSE cur_vencidos;

    SELECT * FROM tmp_atrasos_processados;
END$$
```

C1-antes Antes: pendentes com prazo vencido (o que o cursor vai percorrer)

```
SELECT u.nome AS usuario, l.titulo, e.data_devolucao_prevista, e.status
FROM emprestimos e
JOIN usuarios u ON u.id_usuario = e.id_usuario
JOIN livros   l ON l.id_livro   = e.id_livro
WHERE e.status = 'pendente'
  AND e.data_devolucao_prevista < CURDATE()
ORDER BY e.data_devolucao_prevista;

SELECT status, COUNT(*) AS qtd FROM emprestimos GROUP BY status ORDER BY status;

SELECT COUNT(*) AS registros_no_log FROM log_auditoria;
```

usuario	titulo	data_devolucao_prevista	status
Carla Mendes	O Cortico	2026-10-16	pendente
Felipe Santos	Capitaes da Areia	2026-10-19	pendente
Hugo Pereira	Iracema	2026-10-20	pendente

3 linha(s)

status	qtd
pendente	19
atrasado	11
devolvido	18

3 linha(s)

registros_no_log
3

1 linha(s)

C1-valido Valido: executar a rotina com a data de hoje

```
CALL sp_atualizar_atrasos(NULL, @qtd, @multa_total);
SELECT @qtd AS emprestimos_atualizados, @multa_total AS multa_total_estimada;
```

usuario	livro	prevista	dias_atraso	multa
Carla Mendes	O Cortico	2026-10-16	5	7.50
Felipe Santos	Capitaes da Areia	2026-10-19	2	3.00
Hugo Pereira	Iracema	2026-10-20	1	1.50

3 linha(s)

emprestimos_atualizados	multa_total_estimada
3	12.00

1 linha(s)

C1-depois Depois: nenhum pendente vencido; atrasados aumentaram; log por linha

```
SELECT COUNT(*) AS pendentes_vencidos
FROM emprestimos
WHERE status = 'pendente'
  AND data_devolucao_prevista < CURDATE();

SELECT status, COUNT(*) AS qtd FROM emprestimos GROUP BY status ORDER BY status;

SELECT id_log, LEFT(id_registro, 8) AS id_registro, operacao, valor_antigo, valor_novo
FROM log_auditoria
ORDER BY id_log DESC
LIMIT 3;
```

pendentes_vencidos
0

1 linha(s)

status	qtd
pendente	16
atrasado	14
devolvido	18

3 linha(s)

id_log	id_registro	operacao	valor_antigo	valor_novo
6	03a3a209	UPDATE	status=pendente; prevista=2026-10-20; devolucao=-	status=atrasado; prevista=2026-10-20; devolucao=-
5	03a39fdf	UPDATE	status=pendente; prevista=2026-10-19; devolucao=-	status=atrasado; prevista=2026-10-19; devolucao=-
4	03a39b93	UPDATE	status=pendente; prevista=2026-10-16; devolucao=-	status=atrasado; prevista=2026-10-16; devolucao=-

3 linha(s)

Antes havia 3 pendentes vencidos; depois, nenhum. Cada linha processada gerou um registro no log.

4. Triggers

Trigger	Quando	O que faz
`trg_emprestimos_before_insert`	BEFORE INSERT	Valida prazo e exemplar disponível
`trg_emprestimos_after_insert`	AFTER INSERT	Baixa 1 exemplar
`trg_emprestimos_before_update`	BEFORE UPDATE	Impede reabrir; preenche data de devolução
`trg_emprestimos_after_update`	AFTER UPDATE	Devolve 1 exemplar
`trg_emprestimos_before_delete`	BEFORE DELETE	Impede excluir empréstimo aberto
`trg_livros_before_update`	BEFORE UPDATE (livros)	Impede preço negativo e estoque inválido
`trg_auditoria_*` (4)	AFTER	Gravam OLD/NEW no log

Os testes usam `INSERT` / `UPDATE` / `DELETE` direto, sem procedures: as regras valem para qualquer origem. O código completo das 10 triggers está em `04_triggers.sql`.

4.1 INSERT – validar, baixar estoque e auditar

```
CREATE TRIGGER trg_emprestimos_before_insert
BEFORE INSERT ON emprestimos
FOR EACH ROW
BEGIN
    DECLARE v_disponivel INT;
    DECLARE v_titulo     VARCHAR(255);
    DECLARE v_msg        VARCHAR(255);

    SELECT qtd_disponivel, titulo
      INTO v_disponivel, v_titulo
    FROM livros
    WHERE id_livro = NEW.id_livro;

    IF NEW.data_devolucao_prevista ⩽ NEW.data_saida THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'A devolucao prevista deve ser posterior a data de saida';
    ELSEIF DATEDIFF(NEW.data_devolucao_prevista, NEW.data_saida) > 30 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Prazo maximo de emprestimo e de 30 dias';
    ELSEIF NEW.status <> 'devolvido' AND v_disponivel ⩽ 0 THEN
        SET v_msg = CONCAT('Nao ha exemplares disponiveis de "', v_titulo, '"');
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_msg;
    END IF;
END$$
```

```
CREATE TRIGGER trg_emprestimos_after_insert
AFTER INSERT ON emprestimos
FOR EACH ROW
BEGIN
    IF NEW.status <> 'devolvido' THEN
        UPDATE livros
        SET qtd_disponivel = qtd_disponivel - 1
        WHERE id_livro = NEW.id_livro;
```

```
    END IF;
END$$
```

T1-inv-estoque Invalido: "Lavoura Arcaica" (1 exemplar, ja emprestado)

```
SELECT titulo, qtd_exemplares, qtd_disponivel FROM livros WHERE titulo = 'Lavoura Arcaica';

INSERT INTO emprestimos (id_livro, id_usuario, data_saida, data_devolucao_prevista)
VALUES (@livro_lavoura, @rafael,
        CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY));
```

titulo	qtd_exemplares	qtd_disponivel
Lavoura Arcaica	1	0

1 linha(s)

```
× Error Code: 1644. Nao ha exemplares disponiveis de "Lavoura Arcaica"
```

T2-valido Valido: INSERT direto → estoque baixa e log registra

```
SET @id_teste = UUID();

INSERT INTO emprestimos (id_emprestimo, id_livro, id_usuario, data_saida, data_devolucao_prevista)
VALUES (@id_teste, @livro_olhai, @rafael, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY));

SELECT titulo, qtd_exemplares, qtd_disponivel
FROM livros WHERE titulo = 'Olhai os Lirios do Campo';

SELECT id_log, LEFT(id_registro, 8) AS id_registro, operacao, valor_antigo, valor_novo
FROM log_auditoria ORDER BY id_log DESC LIMIT 1;
```

```
✓ INSERT: 1 linha(s) afetada(s)
```

titulo	qtd_exemplares	qtd_disponivel
Olhai os Lirios do Campo	2	1

1 linha(s)

id_log	id_registro	operacao	valor_antigo	valor_novo
7	04003b73	INSERT	*NULL*	status=pendente; prevista=2026-10-28

1 linha(s)

Um único INSERT disparou três triggers: validação, baixa de estoque e log.

4.2 UPDATE – devolução

```
CREATE TRIGGER trg_emprestimos_before_update
BEFORE UPDATE ON emprestimos
FOR EACH ROW
BEGIN
    IF OLD.status = 'devolvido' AND NEW.status <> 'devolvido' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Emprestimo ja devolvido nao pode ser reaberto';
    END IF;

    IF NEW.status = 'devolvido' AND NEW.data_devolucao_real IS NULL THEN
        SET NEW.data_devolucao_real = CURDATE();
    END IF;
```

```
    IF NEW.data_devolucao_real < NEW.data_saida THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Data de devolucao anterior a data de saida';
    END IF;
END$$
```

```
CREATE TRIGGER trg_auditoria_emprestimos_update
AFTER UPDATE ON emprestimos
FOR EACH ROW
BEGIN
    INSERT INTO log_auditoria (tabela, id_registro, operacao, valor_antigo, valor_novo, usuario_bd)
    VALUES ('emprestimos', NEW.id_emprestimo, 'UPDATE',
            CONCAT('status=', OLD.status, '; prevista=', OLD.data_devolucao_prevista,
                   '; devolucao=', IFNULL(OLD.data_devolucao_real, '-')),
            CONCAT('status=', NEW.status, '; prevista=', NEW.data_devolucao_prevista,
                   '; devolucao=', IFNULL(NEW.data_devolucao_real, '-')),
            USER());
END$$
```

T3-valido **Valido: devolucao sem informar a data → BEFORE preenche, AFTER devolve estoque**

```
UPDATE emprestimos SET status = 'devolvido' WHERE id_emprestimo = @id_teste;

SELECT data_saida, data_devolucao_real, status
FROM emprestimos WHERE id_emprestimo = @id_teste;

SELECT titulo, qtd_exemplares, qtd_disponivel
FROM livros WHERE titulo = 'Olhai os Lirios do Campo';

SELECT id_log, LEFT(id_registro, 8) AS id_registro, operacao, valor_antigo, valor_novo
FROM log_auditoria ORDER BY id_log DESC LIMIT 1;
```

✓ UPDATE: 1 linha(s) afetada(s)

data_saida	data_devolucao_real	status
2026-10-21	2026-10-21	devolvido

1 linha(s)

titulo	qtd_exemplares	qtd_disponivel
Olhai os Lirios do Campo	2	2

1 linha(s)

id_log	id_registro	operacao	valor_antigo	valor_novo
8	04003b73	UPDATE	status=pendente; prevista=2026-10-28; devolucao=-	status=devolvido; prevista=2026-10-28; devolucao=2026-10-21

1 linha(s)

O UPDATE só mudou o status; a data foi preenchida pela BEFORE e o estoque voltou pela AFTER.

T3-inv-reabrir **Invalido: reabrir emprestimo devolvido**

```
UPDATE emprestimos SET status = 'pendente' WHERE id_emprestimo = @id_teste;
```

× Error Code: 1644. Emprestimo ja devolvido nao pode ser reaberto

4.3 DELETE e livros

T4-inv-aberto Invalido: excluir emprestimo em aberto (Ana, "Lavoura Arcaica")

```
SET @id_aberto = (SELECT e.id_emprestimo
                  FROM emprestimos e
                  JOIN livros l ON l.id_livro = e.id_livro
                  WHERE l.titulo = 'Lavoura Arcaica' AND e.status <> 'devolvido');

DELETE FROM emprestimos WHERE id_emprestimo = @id_aberto;
```

```
× Error Code: 1644. Nao e permitido excluir emprestimo em aberto
```

T5-inv-preco Invalido: preco negativo

```
UPDATE livros SET preco = -10.00 WHERE titulo = 'Dom Casmurro';
```

```
× Error Code: 1644. O preco nao pode ser negativo
```

5. Tabela de auditoria (log)

```
CREATE TABLE log_auditoria (
    id_log       INT                              AUTO_INCREMENT PRIMARY KEY,
    tabela       VARCHAR(30)                      NOT NULL,
    id_registro  CHAR(36)                         NOT NULL,
    operacao     ENUM('INSERT', 'UPDATE', 'DELETE') NOT NULL,
    valor_antigo VARCHAR(500),
    valor_novo   VARCHAR(500),
    usuario_bd   VARCHAR(100)                     NOT NULL,
    data_hora    DATETIME                         NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

T6-log Log de auditoria completo

```
SELECT id_log, tabela, LEFT(id_registro, 8) AS id_registro, operacao,
       valor_antigo, valor_novo, DATE_FORMAT(data_hora, '%d/%m %H:%i') AS data_hora
FROM log_auditoria
ORDER BY id_log;
```

id_log	tabela	id_registro	operacao	valor_antigo	valor_novo	data_hora
1	emprestimos	03f773ca	INSERT	*NULL*	status=pendente; prevista=2026-11-04	21/10 23:27
2	emprestimos	039f358a	UPDATE	status=atrasado; prevista=2026-09-27; devolucao=-	status=devolvido; prevista=2026-09-27; devolucao=2026-10-21	21/10 23:27
3	emprestimos	03f773ca	UPDATE	status=pendente; prevista=2026-11-04; devolucao=-	status=pendente; prevista=2026-11-18; devolucao=-	21/10 23:27
4	emprestimos	03a39b93	UPDATE	status=pendente; prevista=2026-10-16; devolucao=-	status=atrasado; prevista=2026-10-16; devolucao=-	21/10 23:27
5	emprestimos	03a39fdf	UPDATE	status=pendente; prevista=2026-10-19; devolucao=-	status=atrasado; prevista=2026-10-19; devolucao=-	21/10 23:27
6	emprestimos	03a3a209	UPDATE	status=pendente; prevista=2026-10-20; devolucao=-	status=atrasado; prevista=2026-10-20; devolucao=-	21/10 23:27
7	emprestimos	04003b73	INSERT	*NULL*	status=pendente; prevista=2026-10-28	21/10 23:27
8	emprestimos	04003b73	UPDATE	status=pendente; prevista=2026-10-28; devolucao=-	status=devolvido; prevista=2026-10-28; devolucao=2026-10-21	21/10 23:27
9	emprestimos	04003b73	DELETE	status=devolvido; saida=2026-10-21; devolucao=2026-10-21	*NULL*	21/10 23:27
10	livros	039a5cd8	UPDATE	preco=29.90	preco=34.90	21/10 23:27

10 linha(s)

6. Rotina completa

Rafael pega *Dois Irmãos* (1 exemplar). Sofia tenta pegar o mesmo livro e é bloqueada. Rafael devolve e a Sofia consegue.



R3 Passo 3 -- Rafael pega o livro (procedure + 3 triggers)

```
CALL sp_registrar_emprestimo('rafael.nascimento@email.com', 'Dois Irmaos', 10, @id_rafael);
SELECT @id_rafael AS id_emprestimo;
```

```
✓ CALL: executado com sucesso
```

id_emprestimo
04050804-cda7-11f1-8a73-26a4eaeb17d5

1 linha(s)

R4 Passo 4 -- Estoque baixou e o log registrou o INSERT

```
SELECT titulo, qtd_exemplares, qtd_disponivel FROM livros WHERE titulo = 'Dois Irmaos';

SELECT id_log, LEFT(id_registro, 8) AS id_registro, operacao, valor_novo
FROM log_auditoria ORDER BY id_log DESC LIMIT 1;
```

titulo	qtd_exemplares	qtd_disponivel
Dois Irmaos	1	0

1 linha(s)

id_log	id_registro	operacao	valor_novo
11	04050804	INSERT	status=pendente; prevista=2026-10-31

1 linha(s)

R5 Passo 5 -- Sofia tenta pegar o mesmo livro: a trigger bloqueia

```
CALL sp_registrar_emprestimo('sofia.cardoso@email.com', 'Dois Irmaos', 10, @id_sofia);
```

```
× Error Code: 1644. Nao ha exemplares disponiveis de "Dois Irmaos"
```

R6 Passo 6 -- Rafael devolve (OUT com a multa)

```
CALL sp_registrar_devolucao(@id_rafael, @multa);
SELECT @multa AS multa;
```

```
✓ CALL: executado com sucesso
```

multa
0.00

1 linha(s)

R8 Passo 8 -- Agora o exemplar voltou: Sofia consegue pegar

```
CALL sp_registrar_emprestimo('sofia.cardoso@email.com', 'Dois Irmaos', 10, @id_sofia);
SELECT titulo, qtd_exemplares, qtd_disponivel FROM livros WHERE titulo = 'Dois Irmaos';
```

✓ CALL: executado com sucesso

titulo	qtd_exemplares	qtd_disponivel
Dois Irmaos	1	0

1 linha(s)

7. Objetos no banco

S1 Procedures criadas

```
SHOW PROCEDURE STATUS WHERE Db = 'biblioteca';
```

Db	Name	Type	Created
biblioteca	cadastrar_usuario	PROCEDURE	2026-10-21 23:27:42
biblioteca	sp_atualizar_atrasos	PROCEDURE	2026-10-21 23:27:43
biblioteca	sp_registrar_devolucao	PROCEDURE	2026-10-21 23:27:43
biblioteca	sp_registrar_emprestimo	PROCEDURE	2026-10-21 23:27:43
biblioteca	sp_relatorio_emprestimos_usuario	PROCEDURE	2026-10-21 23:27:43
biblioteca	sp_renovar_emprestimo	PROCEDURE	2026-10-21 23:27:43
biblioteca	sp_resumo_usuario	PROCEDURE	2026-10-21 23:27:43

7 linha(s)

S2 Triggers criadas

```
SHOW TRIGGERS FROM biblioteca;
```

Trigger	Event	Table	Timing
trg_emprestimos_before_insert	INSERT	emprestimos	BEFORE
trg_emprestimos_after_insert	INSERT	emprestimos	AFTER
trg_auditoria_emprestimos_insert	INSERT	emprestimos	AFTER
trg_emprestimos_before_update	UPDATE	emprestimos	BEFORE
trg_emprestimos_after_update	UPDATE	emprestimos	AFTER
trg_auditoria_emprestimos_update	UPDATE	emprestimos	AFTER
trg_emprestimos_before_delete	DELETE	emprestimos	BEFORE
trg_auditoria_emprestimos_delete	DELETE	emprestimos	AFTER
trg_livros_before_update	UPDATE	livros	BEFORE
trg_auditoria_livros_update	UPDATE	livros	AFTER

10 linha(s)